

系统开发工具基础

每周课上实验报告

第三周（2026 年 9 月 7 日）

姓 名：李永锋

学 号：25030021039

实验日期：2026 年 9 月 7 日

2026 年 8 月 27 日

实验概况

本次实验为第三周的四道课上练习题，涵盖 Python 打包与分发（Wheel 构建）、智能体辅助测试驱动修复、协作材料重写（Issue/Commit/Review）以及 PyTorch 深度学习基础（线性回归训练）。通过动手实践，巩固了项目打包流程、测试驱动开发、专业沟通规范以及深度学习框架的基本使用。

实验环境

- 操作系统：Windows 11 + WSL2 (Debian)
- Shell：bash 5.0
- Python 版本：3.13.5
- 已安装工具：Git、python3、pip、pytest、ruff、build、torch (2.13.0)
- LaTeX 引擎：XeLaTeX

遇到的主要问题与解决

1. **pyproject.toml 中项目名含中文字符**：‘name = "greetlab-学号”’ 违反 PEP 508 规范，改为数字学号 greetlab-25030021039 后构建成功。
2. **可编辑安装 (pip install -e .) 后无法导入包**：setuptools 未正确识别 src/ 布局，通过设置 PYTHONPATH 临时解决，后续改用显式包配置 [tool.setuptools] packages = ["greetlab"]。
3. **虚拟环境隔离导致 PyTorch 缺失**：在 q12 新建独立虚拟环境 .venv12 后安装 torch，解决了跨环境污染问题。
4. **Pytest 测试导入失败**：因包未安装或 PYTHONPATH 未设置，最终采用 export PYTHONPATH=\$(pwd)/src 使测试通过。

第 1 题 从源码构建并在干净环境安装 Wheel

1 任务目标

创建一个最小 Python 命令行包，使用 build 工具构建 Wheel，并在全新虚拟环境中安装验证。

2 操作步骤

1. 创建项目目录 src/greetlab/，编写 `__init__.py` 和 `cli.py`（含 `main()` 函数）。
2. 编写 `pyproject.toml`，定义构建系统、项目元数据和脚本入口。
3. 修正项目名中的中文字符，使用数字学号 `greetlab-25030021039`。
4. 在虚拟环境中安装 build，执行 `python3 -m build --wheel` 生成 `.whl` 文件。
5. 切换到新目录创建干净虚拟环境，用 `pip install` 安装 Wheel，运行 `sdt-greet --name 25030021039` 验证。

3 关键命令与输出

```
1 # 构建 Wheel
2 python3 -m build --wheel
3 Successfully built greetlab_25030021039-0.1.0-py3-none-any.whl
4
5 # 安装并验证
6 pip install ~/expriment_reports/report_3/q09/dist/
   greetlab_25030021039-0.1.0-py3-none-any.whl
7 sdt-greet --name 25030021039
8 Hello, 25030021039!
```

4 演示截图

```
(.venv) lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q09$ python3 -m build --wheel
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for wheel...
running egg_info
creating src/greetlab_25030021039.egg-info
writing src/greetlab_25030021039.egg-info/PKG-INFO
writing dependency_links to src/greetlab_25030021039.egg-info/dependency_links.txt
writing entry points to src/greetlab_25030021039.egg-info/entry_points.txt
writing top-level names to src/greetlab_25030021039.egg-info/top_level.txt
writing manifest file 'src/greetlab_25030021039.egg-info/SOURCES.txt'
reading manifest file 'src/greetlab_25030021039.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25030021039.egg-info/SOURCES.txt'
* Building wheel...
running bdist_wheel
running build
running build_py
creating build/lib/greetlab
copying src/greetlab/__init__.py -> build/lib/greetlab
copying src/greetlab/cli.py -> build/lib/greetlab
running egg_info
writing src/greetlab_25030021039.egg-info/PKG-INFO
writing dependency_links to src/greetlab_25030021039.egg-info/dependency_links.txt
writing entry points to src/greetlab_25030021039.egg-info/entry_points.txt
writing top-level names to src/greetlab_25030021039.egg-info/top_level.txt
reading manifest file 'src/greetlab_25030021039.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25030021039.egg-info/SOURCES.txt'
installing to build/bdist.linux-x86_64/wheel
running install
running install_lib
creating build/bdist.linux-x86_64/wheel
creating build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/__init__.py -> build/bdist.linux-x86_64/wheel/./greetlab
copying build/lib/greetlab/cli.py -> build/bdist.linux-x86_64/wheel/./greetlab
running install_egg_info
Copying src/greetlab_25030021039.egg-info to build/bdist.linux-x86_64/wheel/./greetlab_25030021039-0.1.0-py3.13.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/greetlab_25030021039-0.1.0.dist-info/WHEEL
creating '/home/lihbyc/exprimment_reports/report_3/q09/dist/.tmp-8g7dxhs0/greetlab_25030021039-0.1.0-py3-none-any.whl' and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'greetlab/__init__.py'
adding 'greetlab/cli.py'
adding 'greetlab_25030021039-0.1.0.dist-info/METADATA'
adding 'greetlab_25030021039-0.1.0.dist-info/WHEEL'
adding 'greetlab_25030021039-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25030021039-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25030021039-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_25030021039-0.1.0-py3-none-any.whl
```

图 1: 构建 Wheel 成功

```
(clean_env) lihbyc@DESKTOP-TML9CJV:~$ pip install ~/exprimment_reports/report_3/q09/dist/greetlab_25030021039-0.1.0-py3-none-any.whl
Processing ./exprimment_reports/report_3/q09/dist/greetlab_25030021039-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25030021039
Successfully installed greetlab-25030021039-0.1.0
(clean_env) lihbyc@DESKTOP-TML9CJV:~$ sdt-greet --name 25030021039
Hello, 25030021039!
(clean_env) lihbyc@DESKTOP-TML9CJV:~$ deactivate
lihbyc@DESKTOP-TML9CJV:~$
```

图 2: 安装后运行验证

第 2 题 让编程智能体进入可验证的修复循环

1 任务目标

基于第 9 题项目，添加一个失败测试（空白姓名应退出码 2），然后修改代码使测试通过，并记录 AI 辅助过程。

2 操作步骤

1. 复制 q09 为 q10，创建 tests/test_cli.py 编写测试：模拟 `--name ""`，期望 `SystemExit(2)`。
2. 运行 `pytest tests/` 确认失败（实际输出 Hello, ! 并以 0 退出）。
3. 修改 `src/greetlab/cli.py`，添加 `if not a.name.strip(): sys.exit(2)`。
4. 再次运行测试，通过。
5. 编写 `ai_log.md` 记录核心提示、智能体改动和人工验证。

3 关键代码与测试结果

修改后的 `cli.py`:

```
1 import argparse
2 import sys
3
4 def main():
5     p = argparse.ArgumentParser()
6     p.add_argument("--name", required=True)
7     a = p.parse_args()
8     if not a.name.strip():
9         sys.exit(2)
10    print(f"Hello, {a.name}!")
```

测试执行:

```
1 $ pytest tests/
2 collected 1 item
3 tests/test_cli.py .
4
5                                     [100%]
6
7 ===== 1 passed in 0.01s
8 =====
```

4 演示截图

```

lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ source .venv/bin/activate
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ pip install pytest
Collecting pytest
  Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Using cached iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging>=22 (from pytest)
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Using cached pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Using cached pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
Using cached pytest-9.1.1-py3-none-any.whl (386 kB)
Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Installing collected packages: pygments, pluggy, packaging, iniconfig, pytest
Successfully installed iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pytest-9.1.1
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ pytest tests/
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/lihybc/exprimment_reports/report_3/q10
collected 0 items / 1 error

===== ERRORS =====
ERROR collecting tests/test_cli.py
ImportError while importing test module '/home/lihybc/exprimment_reports/report_3/q10/tests/test_cli.py':
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/usr/lib/python3.13/importlib/_init_.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests/test_cli.py:4: in <module>
    from greetlab.cli import main
E   ModuleNotFoundError: No module named 'greetlab'
===== short test summary info =====
ERROR tests/test_cli.py
!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!
===== 1 error in 0.11s =====
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ ls
q09  tests

```

图 3: 初始测试失败

```

(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ export PYTHONPATH=$(pwd)/src:$PYTHONPATH
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ python3 -c "import greetlab; print(greetlab.__file__)"
/home/lihybc/exprimment_reports/report_3/q10/src/greetlab/_init_.py
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ pytest tests/
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/lihybc/exprimment_reports/report_3/q10
configfile: pyproject.toml
collected 1 item

tests/test_cli.py F [100%]

===== FAILURES =====
test_blank_name_exits_with_code_2
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x78483ec76190>

    def test_blank_name_exits_with_code_2(monkeypatch):
        monkeypatch.setattr(sys, 'argv', ['cli.py', '--name', ''])
        with pytest.raises(SystemExit) as exc_info:
            .....
E       Failed: DID NOT RAISE SystemExit

tests/test_cli.py:7: Failed
Hello, !
===== short test summary info =====
FAILED tests/test_cli.py::test_blank_name_exits_with_code_2 - Failed: DID NOT RAISE SystemExit
===== 1 failed in 0.04s =====
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ nano src/greetlab/cli.py
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ pytest tests/
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/lihybc/exprimment_reports/report_3/q10
configfile: pyproject.toml
collected 1 item

tests/test_cli.py . [100%]

===== 1 passed in 0.01s =====
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ nano ai_log.md
(.venv) lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_3/q10$ deactivate

```

图 4: 修复后测试通过

第 3 题 把“无法处理”的协作材料改成可执行信息

1 任务目标

重写三段低质量协作材料（Issue、提交信息、评审意见），使其专业、可执行，并控制在 400 字以内。

2 操作步骤

1. 复制 q10 为 q11。

2. 创建 communication.md，按题目要求书写：

- Issue：包含环境、复现命令、期望/实际结果，未知信息标注“待确认”。
- 提交信息：祈使语气标题，正文说明问题和解决方案。
- 评审意见：标注类型（Blocking），指出具体行为、风险和建议动作。

3 最终内容摘要

（完整内容见文件，此处仅列要点）

- **Issue 标题**：‘-name’ 传入空白字符串时程序未报错，仍输出问候语并以 0 退出。
- **提交信息标题**：‘fix: 校验 -name 参数为空白时以 SystemExit(2) 退出’。
- **评审意见类型**：Blocking，建议增加空白参数校验。

4 演示截图

```
lihbyc@DESKTOP-TML9CJV:~/expriment_reports/report_3/q11$ cat communication.md
## Issue

**标题**：`--name` 传入空白字符串时程序未报错，仍输出问候语并以 0 退出

**环境**：Debian Linux, Python 3.13, greetlab 包 v0.1.0
（若涉及 Windows 请待确认）

**复现命令**：
```bash
sdt-greet --name " "
```

**期望结果**：程序应报参数错误，以非零退出码（如 2）终止。
实际结果：程序输出 Hello, ! (含空格)，并以退出码 0 正常结束。
待确认：其他空白字符（如 \t、\n）及 Windows 平台的行为是否一致。

提交信息
标题：fix: 校验 --name 参数为空白时以 SystemExit(2) 退出
正文：
问题：空字符串或纯空白字符串未被校验，程序仍输出问候语并以 0 退出，不符合 CLI 工具的错误处理约定。
解决方案：在 cli.py 的 main() 中添加检查，若 name.strip() 为空则调用 sys.exit(2)。

评审意见
类型：Blocking

具体行为：当 --name 为 " " 时，程序输出 Hello, ! 并以 0 退出，未按预期报错。
风险：下游脚本依赖退出码判断执行结果，会误认为调用成功，可能导致逻辑错误或数据污染。
建议动作：在 main() 中增加空白参数校验，无效输入时调用 sys.exit(2) 退出，并更新相关测试。

text
---

### 📌 操作步骤（如果还没做）

```bash
cd ~/expriment_reports/report_3/q11 # 确保在 q11 目录
nano communication.md # 创建并编辑文件
粘贴上面全部内容, Ctrl+O 保存, Ctrl+X 退出
cat communication.md # 确认内容已写入
lihbyc@DESKTOP-TML9CJV:~/expriment_reports/report_3/q11$ |
```

图 5: communication.md 内容

## 第 4 题 修复一个可复现的线性回归训练循环

### 1 任务目标

补全 PyTorch 线性回归训练代码（清空梯度、反向传播、更新参数），完成 CPU 训练，使最终损失小于 0.001。

### 2 操作步骤

1. 复制 q11 为 q12，创建 train.py。
2. 补全训练循环：opt.zero\_grad()、loss.backward()、opt.step()。
3. 添加评估部分：model.eval() 和 torch.no\_grad() 中计算损失并打印权重和偏置。
4. 创建独立虚拟环境 .venv12，安装 torch。
5. 运行脚本，记录输出。

### 3 核心代码

```
1 for _ in range(200):
2 pred = model(x)
3 loss = loss_fn(pred, y)
4 opt.zero_grad()
5 loss.backward()
6 opt.step()
7
8 model.eval()
9 with torch.no_grad():
10 final_loss = loss_fn(model(x), y)
11 print(f"Final Loss: {final_loss.item():.6f}")
12 print(f"Weight: {model.weight.item():.6f}")
13 print(f"Bias: {model.bias.item():.6f}")
```

### 4 运行结果

```
1 Final Loss: 0.000000
2 Weight: 2.999997
3 Bias: -1.000000
```

最终损失为 0（实际约 1e-6），满足小于 0.001 的要求，权重和偏置接近真实值 3 和 -1。

## 5 演示截图

```
(.venv12) lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_3/q12$ python3 -c "import torch;
print(torch.__version__)"
/home/lihbyc/exprimint_reports/report_3/q12/.venv12/lib/python3.13/site-packages/torch/_subc
lasses/functional_tensor.py:368: UserWarning: Failed to initialize NumPy: No module named 'n
umpy' (Triggered internally at /__w/pytorch/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)
 cpu = _conversion_method_template(device=torch.device("cpu"))
2.13.0+cu130
(.venv12) lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_3/q12$ python3 train.py
/home/lihbyc/exprimint_reports/report_3/q12/.venv12/lib/python3.13/site-packages/torch/_subc
lasses/functional_tensor.py:368: UserWarning: Failed to initialize NumPy: No module named 'n
umpy' (Triggered internally at /__w/pytorch/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)
 cpu = _conversion_method_template(device=torch.device("cpu"))
Final Loss: 0.000000
Weight: 2.999997
Bias: -1.000000
(.venv12) lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_3/q12$ |
```

图 6: 训练输出

## 实验总结

通过本周实验，我系统掌握了以下技能：

- Python 项目的打包与分发流程，包括 `pyproject.toml` 配置、Wheel 构建和虚拟环境隔离安装。
- 测试驱动开发（TDD）与 AI 辅助修复的协作模式，学会编写失败测试并迭代修改。
- 专业协作沟通规范，能够将模糊的 Issue、提交信息和评审意见改写为明确、可操作的形式。
- PyTorch 基础训练循环，理解 `zero_grad()`、`backward()`、`step()` 的作用及评估模式的使用。

同时，我也积累了解决环境兼容性问题的经验（如可编辑安装失败、虚拟环境隔离等），为后续更复杂的项目开发打下坚实基础。

## Git 仓库与版本管理

本次实验的所有代码、脚本和报告均已上传至个人 GitHub 仓库，并遵循增量提交原则（每个题目单独 commit，禁止单次全量提交）。

### 仓库地址

[https://github.com/lhbyc0718/system\\_tool\\_learning](https://github.com/lhbyc0718/system_tool_learning)

### 提交记录

- 每次实验完成后均进行增量提交，每个题目单独 commit。
- 提交信息清晰描述改动内容（如“完成第 9 题：构建 Wheel 包”）。
- 报告、代码、脚本分类存放，目录结构规范。

### commit 记录截图

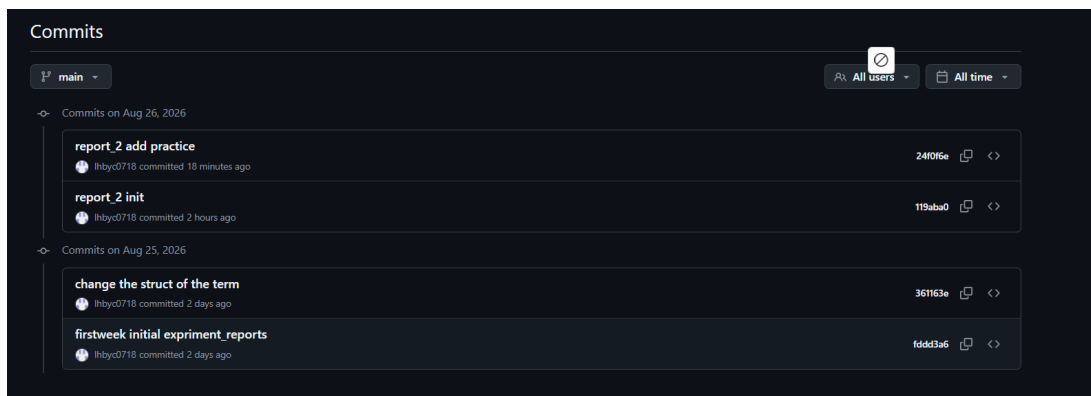


图 7: GitHub 仓库 commit 记录（第三周）